# Meridian: Latent Cartographer — A Local-First Spatial Knowledge Engine

Project Proposal for UCS503P
Submitted to: Dr. Jeelani Asif
**Thapar Institute of Engineering and Technology**

Rahul Dadwal, CSED* Jyotsna Sachdeva, CSED† Bisman Singh Rai, CSED‡

August 26, 2026

## 1 Higher-order goal

This project aims to enhance digital research productivity and knowledge synthesis by reducing cognitive load and context fragmentation during complex web sessions. While digital information management spans diverse personal knowledge workflows, the immediate engineering objective is to translate *on-device spatial vector organization and semantic retrieval* into a measurable, responsive, and privacy-preserving browser extension.

## 2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core DOM-to-canvas rendering and vector indexing workflow, validating cluster navigation within an initial 1–2 week iteration.
- We will prioritize fast feedback over perfection by utilizing modular WebAssembly (WASM) ML inference workers and offscreen pipelines with CI-backed automation to minimize integration overhead.
- Success is defined by achieving a functioning 2D interactive canvas with on-device semantic search in the first iteration, progressively refining tiered caching and layout physics in subsequent iterations.

## 3 Problem Statement

Developers, researchers, and students routinely experience severe cognitive overload and context loss when conducting research across dozens of open tabs, PDFs, and code references. Key pain points include:

- **Fragmentation & Tab Overload:** Information is scattered across flat, linear tab bars where titles and context are clipped.
- **Semantic Drift & Shallow Indexing:** Standard bookmarking tools either store static URLs or rely on crude first-paragraph text matching, failing to capture embedded code snippets, tables, and granular knowledge.

*Roll No: 1024160014, Email: rdadwal_be24@thapar.edu
†Roll No: 1024160133, Email: jsachdeva_be24@thapar.edu
‡Roll No: 1024160130, Email: brai_be24@thapar.edu

- **High Latency & Privacy Vulnerabilities:** Cloud-dependent semantic search engines introduce network lag, API cost overhead, and expose sensitive browsing history to external servers.
- **Memory Inefficiency:** Accumulating unmanaged screenshots and raw DOM snapshots exhausts browser memory and IndexedDB storage quotas.

**Why this matters now (contextual relevance):** In an era of intense information density, providing a zero-cost, local-first visual memory map directly inside the browser allows users to resume research workflows with minimal cognitive friction and total data privacy.

# 4 Proposed Solution

## 4.1 Overview

Build **Meridian: Latent Cartographer**, a client-side Chromium extension (Manifest V3) that organizes digital artifacts on an infinite 2D canvas:

- **Multi-Source Ingestion Pipeline:** Ingests active web tabs, local PDFs (via `pdf.js`), Markdown notes, and code snippets.
- **Local-First AI Engine:** Executes on-device feature extraction using `Xenova/all-MiniLM-L6-v2` (ONNX/WASM) in dedicated Web Workers to generate 384-dimensional dense vectors.
- **Interactive 2D Spatial Canvas:** Renders a 60 FPS spatial graph using HTML5 Canvas / Konva.js with force-directed semantic clustering.
- **Sub-20ms Intent Search:** Real-time cosine similarity search navigating the canvas directly to relevant concept clusters.
- **Tiered Storage Hierarchy:** Hybrid storage leveraging IndexedDB for quantized vectors and Origin Private File System (OPFS) for heavy binary screenshots.

## 4.2 Core workflow

The end-to-end operational execution is structured around an asynchronous multi-stage tree pipeline mapping raw digital artifacts to an on-device 2D vector landscape (Figure 1):

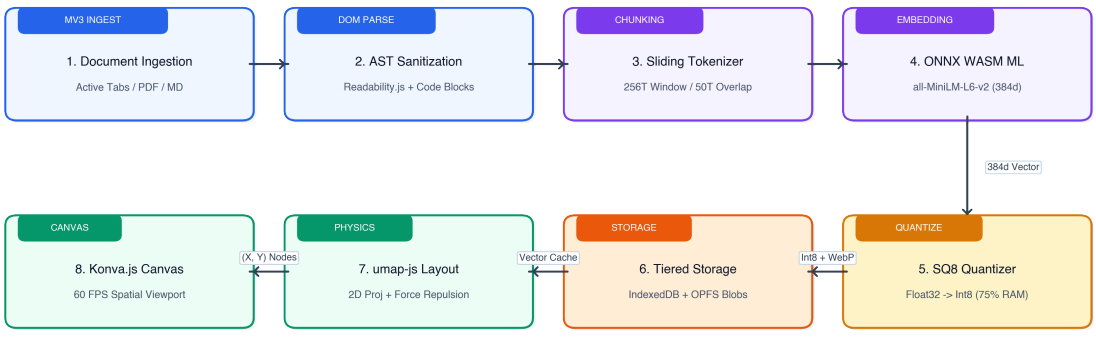


**Figure 1:** Squarish Pipeline Architecture: Modular tree-fork flow showing DOM sanitization, sliding-window chunking, on-device WASM embedding, SQ8 quantization, and 60 FPS canvas projection.

### 4.3 Operational constraints for an educational, local-first setting

- Strictly 100% client-side execution; no reliance on paid external cloud APIs or proprietary LLM endpoints.
- Adherence to Chromium Manifest V3 constraints (service worker termination handling via offscreen documents/IndexedDB persistence).
- Memory-bounded operation through vector quantization (SQ8) and canvas viewport culling.

## 5 Solution Approach

This is an engineering course project focusing on robust local-first systems architecture, data pipeline engineering, and responsive UI rendering rather than novel neural network design.

### 5.1 Knowledge extraction & heuristic clustering

Data extraction and grouping rely on established engineering primitives:

- **Hierarchical Chunking:** Sliding-window tokenization (256-token windows, 50-token overlap) to index granular sections.
- **DOM AST Isolation:** Targeted parsing of `<pre><code>`, `<table>`, and `<h1>--<h3>` tags.
- **Clustering & Layout:** Vector similarity projection (cosine distance-seeded spring-force layout) executed off-thread to prevent UI blocking.

### 5.2 Extension architecture & Intent Search Flowchart

A decoupled local-first architecture separates compute-intensive ML inference, high-throughput graphics rendering, and persistent storage across isolated threads:

- **Frontend Canvas:** React + Vite + Konva.js rendering responsive cards, search overlays, and cluster boundaries at 60 FPS.
- **Background Worker (MV3):** Manages tab capture events, viewport screenshots, and offscreen worker dispatching.
- **Inference Web Worker:** Encapsulates `@huggingface/transformers` WASM engine for non-blocking embedding generation.
- **Tiered Storage Engine:** IndexedDB (hot layer for SQ8 vectors and metadata) and OPFS (cold layer for WebP snapshots) with an automated LRU cache eviction policy.

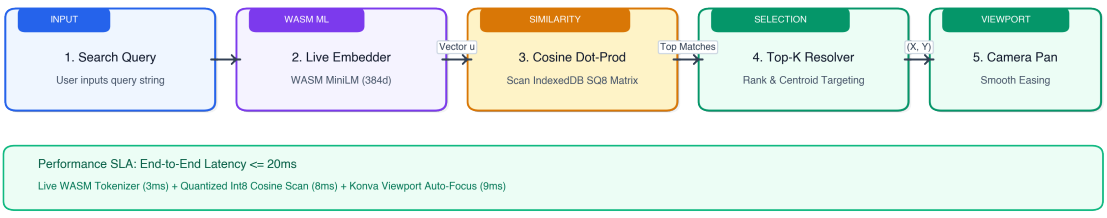


**Figure 2:** Real-Time Search Retrieval Tree: 4-tier fork sequence covering live WASM embedding, fast SQ8 cosine scoring, smooth viewport easing ($\leq$ 20ms), and on-demand OPFS asset inspection.

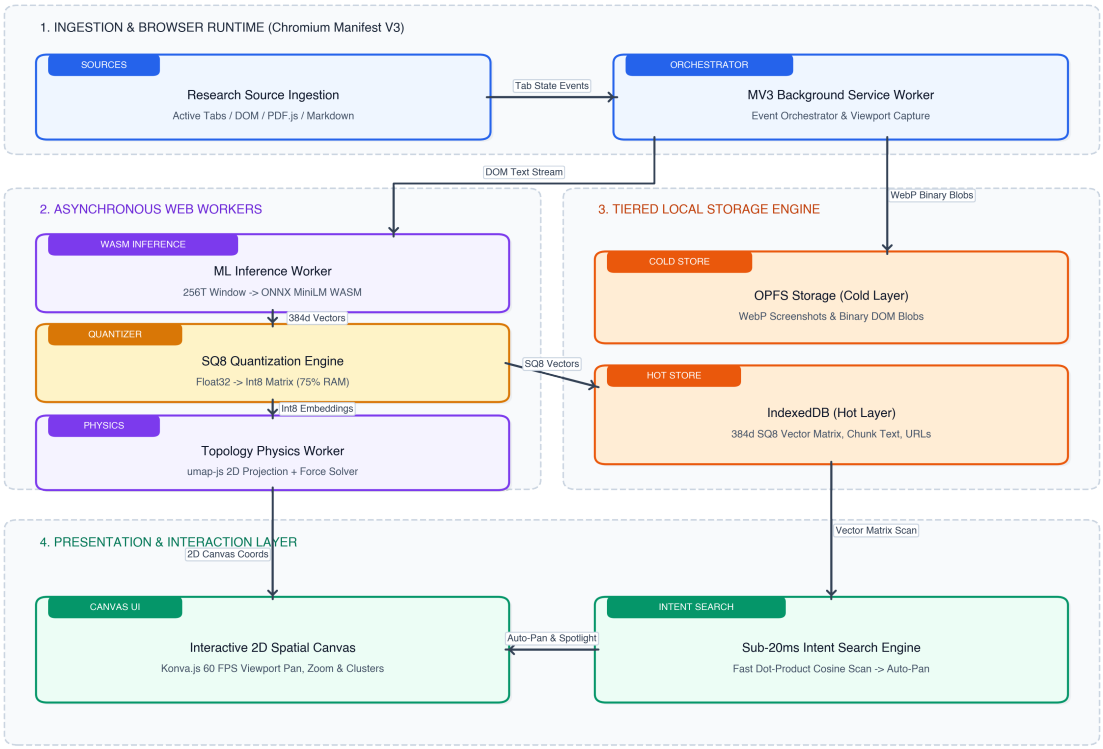


**Figure 3:** Meridian End-to-End System Architecture: decoupled Chromium MV3 runtime with dedicated ML Web Workers, tiered local storage, and high-performance Konva.js 2D spatial canvas.

## 5.3 System Modeling: Use Cases and Functional Specifications

The functional surface is defined across three primary actors: the *Researcher (User)*, the *Chromium Browser Runtime*, and the *Local File System*.

| ID | Use Case | Functional Specification & Scope |
|---|---|---|
| **UC-1** | Ingest Active Tab | Extracts DOM text via `Readability.js` and captures a WebP thumbnail preview. |
| **UC-2** | Ingest Local PDF/Notes | Parses dropped PDF streams via `pdf.js` or Markdown files into searchable AST blocks. |
| **UC-3** | Toggle Deep Focus Mode | Triggers 256-token sliding-window chunking and explicit `<pre><code>` AST extraction. |
| **UC-4** | Local Vector Generation | Generates 384d dense embeddings via ONNX/WASM Web Workers and executes SQ8 quantization. |
| **UC-5** | Execute Intent Search | Matches natural language queries via Cosine Similarity and centers the viewport in $\leq$ 20ms. |
| **UC-6** | LRU Storage Maintenance | Automated background purge of cold OPFS snapshots ($>$ 3–7 days) while retaining SQ8 vectors. |
| **UC-7** | Export Portable Space | Packages canvas coordinates, metadata, and quantized vectors into a standalone `.html` file. |

## 5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build Vite bundles and run unit tests for vector similarity and parsing functions.

- Validate Manifest V3 compliance and extension packaging on pull requests.
- Enable reproducible, automated builds for local browser testing and staged releases.

# 6 Evaluation Criterion

Success is evaluated via concrete performance, memory, and usability benchmarks.

## 6.1 Primary evaluation metric

**Search Retrieval Latency (SRL):** Time elapsed from query submission to vector calculation, cosine scoring, and canvas focus transition.

- Target: Median SRL $\leq$ 20ms across an index of 200+ indexed artifacts.
- Attribution: Measured via high-resolution browser performance timestamps (`performance.now()`).

## 6.2 Secondary evaluation metrics

- **Canvas Rendering Smoothness:** Maintain $\geq$ 55 FPS during viewport panning and zooming under 100+ active nodes.
- **Storage Footprint Reduction:** Achieve $\geq$ 70% storage savings for vector data via SQ8 quantization compared to raw Float32 representations.
- **Ingestion Processing Time:** Background DOM extraction and vectorization completed within $\leq$ 1.5 seconds per tab.
- **Local Execution Reliability:** 100% offline availability for embedding, indexing, and spatial search.

## 6.3 Pilot validation plan

- Conduct user trials with 10–15 engineering students performing structured literature review and debugging tasks.
- Measure workflow resumption speed and subjective cognitive clarity comparing standard tab bars vs. Meridian spatial clusters.

# 7 Scalability

The system scales both computationally and operationally.

1. **Scalable (theoretically):** The client-side architecture scales as user workspaces grow to hundreds of items by:
   - applying viewport culling and bitmap caching in Konva.js to limit GPU draw overhead,
   - quantizing embedding matrices (SQ8 Int8) to minimize RAM consumption,
   - offloading dimensionality reduction and vector math to background Web Workers.
2. **Operationally deployable (practically):** The solution requires zero cloud infrastructure:
   - distributed directly as a standard, standalone Chromium/Firefox extension bundle,
   - self-contained with pre-quantized ONNX model weights stored locally in browser cache,
   - support for single-file portable HTML/JSON export of canvas workspaces.

# 8 Engine availability heuristic

A mature ecosystem of client-side web technologies is directly available for implementation:

- **Vite + React:** Modern frontend toolchain and component framework.
- **HTML5 Canvas / Konva.js:** Production-ready 2D graphics rendering engine.
- **Transformers.js (ONNX Runtime WASM):** Mature on-device ML execution framework.
- **IndexedDB & OPFS:** Standardized, high-throughput browser storage primitives.
- **Readability.js & PDF.js:** Battle-tested DOM and document parsing engines.
- **GitHub Actions:** Automated CI test runner and extension packager.

# 9 Project scope and deliverables

## 9.1 Initial deliverable

- Chrome Extension skeleton (MV3) with DOM text and tab capture
- Web Worker integration with `Transformers.js` running `all-MiniLM-L6-v2`
- Basic Konva.js 2D canvas displaying captured tab cards
- Sub-20ms cosine similarity query input with card highlight
- Automated CI pipeline for linting, build verification, and unit tests

## 9.2 Subsequent deliverables

- AST code block and Markdown table extraction pipeline
- Dual-tier context mode (Default lightweight vs. Deep Focus mode)
- Tiered storage management (IndexedDB + OPFS) with automated LRU purge
- Multi-source ingestion support for local PDFs and standalone notes
- Single-file standalone HTML export for portable canvas sharing

# 10 Risks and mitigations

- **Manifest V3 Service Worker Inactivity:** Mitigate by offloading persistent tasks to offscreen canvas documents and persisting runtime state to IndexedDB.
- **Browser Memory Overhead:** Mitigate by implementing SQ8 scalar quantization, OPFS screenshot offloading, and aggressive canvas viewport culling.
- **WASM Initialization Latency:** Mitigate by pre-warming model pipelines asynchronously in background workers on browser startup.
- **DOM Noise / Content Quality:** Mitigate by utilizing `Readability.js` heuristics to strip sidebars, navigation bars, and footers prior to tokenization.

## 11 Summary

This project proposes **Meridian**, an on-device spatial knowledge engine that resolves digital context fragmentation by mapping research artifacts onto a dynamic 2D vector topology. By operating 100% locally with WebAssembly, it satisfies key academic and engineering criteria: delivering rapid time-to-value, leveraging mature client-side engines, enforcing strict user data privacy, and providing measurable evaluation metrics centered around search latency and memory efficiency.